



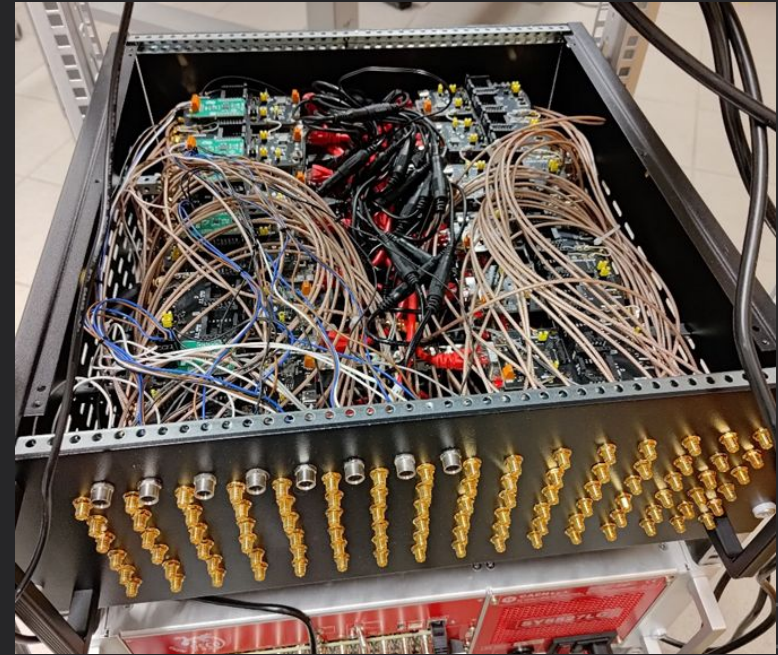
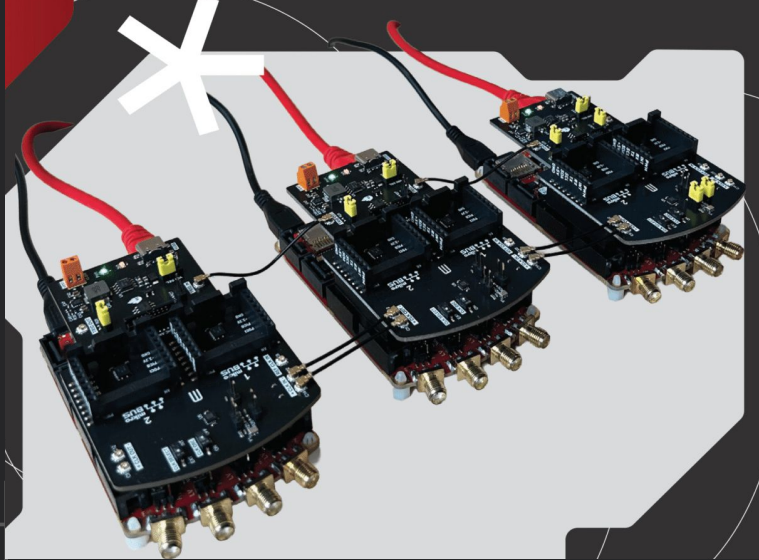
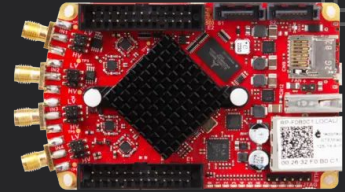
Servidor de Control Centralizado Mediante Protocolo de Acceso Remoto a API de Placas Red Pitayas

Laboratorio de Redes - Instituto Balseiro

Lorenzo Cabrera Blanch - 02/06/26

Clusterizado de HW

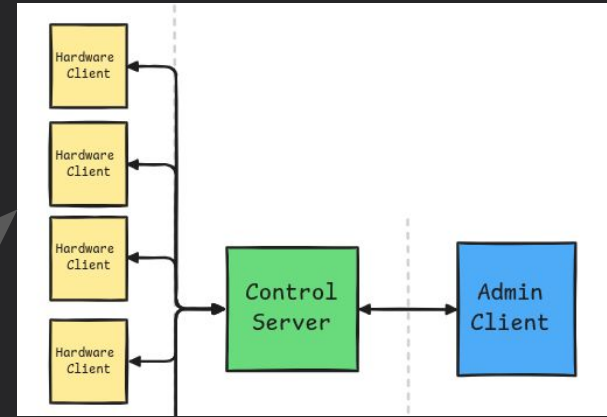
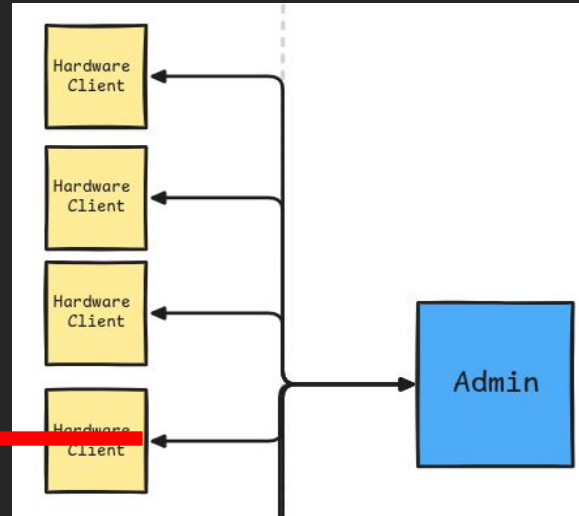
Modelo permite expansión con adquisición
Sincrónica mediante una interconexión en cadena



Cluster de 96 canales (24 pitaya)

Problema de Control Múltiple

- Manejo Manual de múltiples conexiones y contextos
 - Direcciones de conexión
 - Salidas
 - Eventos



Alguien lo tiene que hacer, creamos un **elemento intermedio de control**.

Interacción con el HW

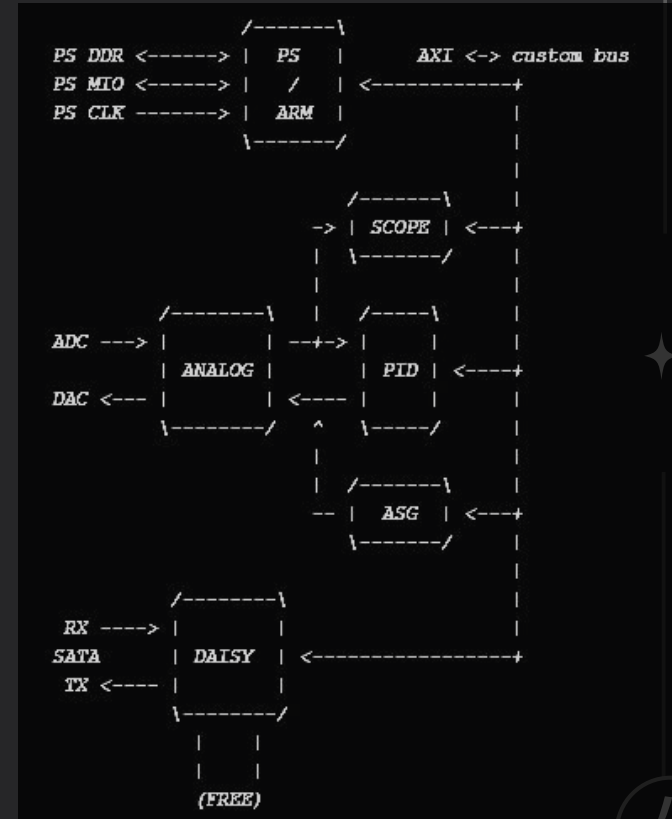
Cada HW contiene funciones a las que podemos **linkear** dinámicamente para acceder a una interfaz de configuración. Esto es lo que se llama una **librería** o **API** (application programming interface).



La API es extensa, esto refleja todos los subsistemas que contiene el HW (FPGA) de la placa de desarrollo.

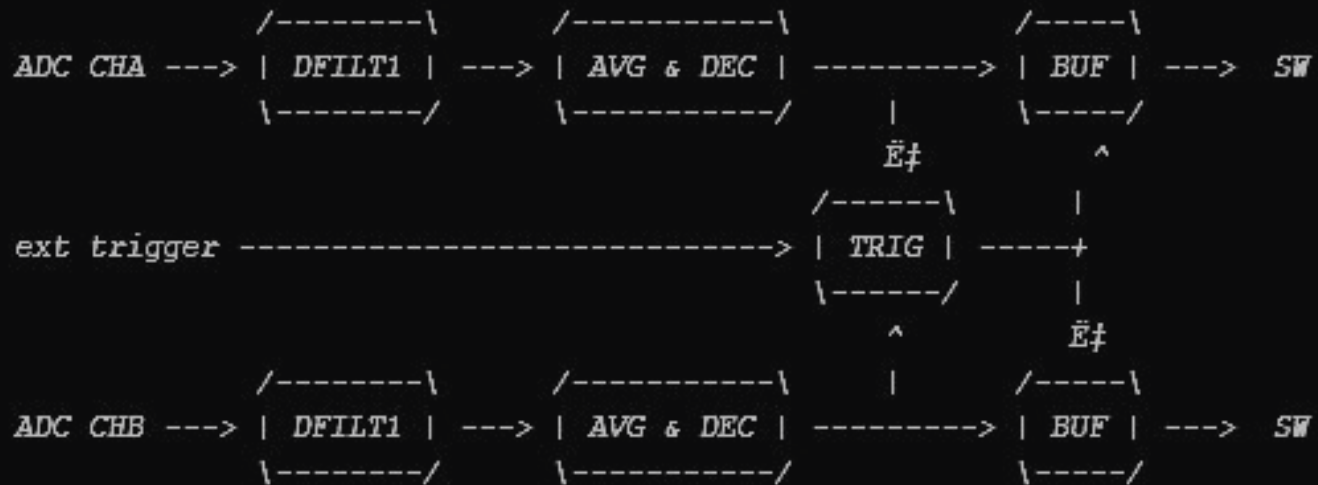
- **Oscilloscope**
- **Arbitrary signal generator (ASG)**
- **PID controller**
- **Mixed analog signals (AMS)**
- **Daisy chaining**

Todos estos se encuentran mapeados por memoria (MMIO).



Ejemplo de Subsistema - Osciloscopio

Subsistema de adquisición de datos por ADC en HW, diagrama en bloques



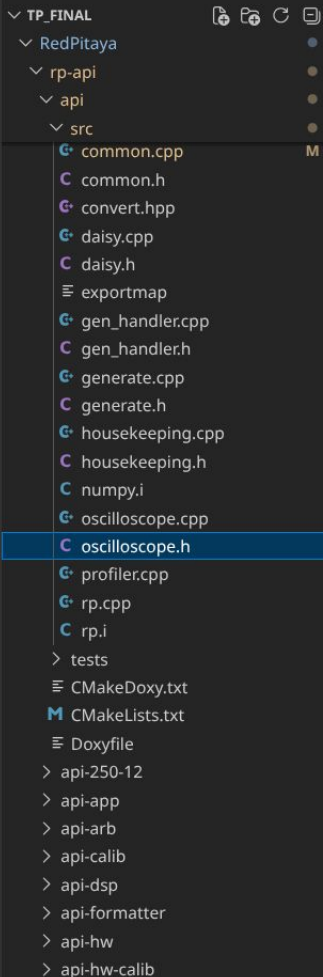
Cada uno de estos bloques tienen elementos de estatus y control

Ej

Código base
de la API de
captura de
datos

✦ (osciloscopio)

Muchísimos
métodos,
macros y
variables que
abstraen la
interacción
con el HW.



```
RedPitaya > rp-api > api > src > C oscilloscope.h > osc_SetUnlockTrigger(rp_channel_t)
11  #ifndef SRC_OSCILLOSCOPE_H_
12
13  // Base Oscilloscope address
14  static const int OSC_BASE_ADDR = 0x00100000;
15  static const int OSC_BASE_SIZE = 0x30000;
16  static const int OSC_BASE_ADDR_4CH = 0x00200000;
17
18  // Oscilloscope Channel A input signal buffer offset
19  #define OSC_CHA_OFFSET 0x10000
20
21  // Oscilloscope Channel B input signal buffer offset
22  #define OSC_CHB_OFFSET 0x20000
23
24  typedef struct {
25      uint8_t start_write : 1;           // (W) start write
26      uint8_t reset_state_machine : 1;  // (W) rst_wr_state_machine
27      uint8_t trigger_status : 1;        // (R) trigger_status
28      uint8_t arm_keep : 1;              // (W) arm_keep
29      uint8_t all_data_written : 1;      // (R) All data written to buffer
30      uint8_t enable_split_trigger : 1;  // Work only first channel
31      uint8_t : 2;
32      void print() volatile {
33          printRegBit(" - %-39s = 0x%08X (%d)\n", "start_write", start_write);
34          printRegBit(" - %-39s = 0x%08X (%d)\n", "reset_state_machine", reset_state_machine);
35          printRegBit(" - %-39s = 0x%08X (%d)\n", "trigger_status", trigger_status);
36          printRegBit(" - %-39s = 0x%08X (%d)\n", "arm_keep", arm_keep);
37          printRegBit(" - %-39s = 0x%08X (%d)\n", "all_data_written", all_data_written);
38          printRegBit(" - %-39s = 0x%08X (%d)\n", "enable_split_trigger", enable_split_trigger);
39      };
40  } config_ch_t;
41
42  typedef struct {
43      config_ch_t config_ch[4];
44  } config_t;
45
46  typedef union {
47      uint32_t reg_full;
48      config_t reg;
49  } config_u_t;
50
```



```

566 static const uint32_t DATA_DEC_MASK = 0x1FFFF; // (17 bits)
567 static const uint32_t DATA_AVG_MASK = 0x1; // (1 bit)
568 static const uint32_t TRIG_SRC_MASK = 0xF; // (4 bits)
569 static const uint32_t START_DATA_WRITE_MASK = 0x1; // (1 bit)
570 static const uint32_t THRESHOLD_MASK = 0xFFFF; // (16 bits)
571 static const uint32_t HYSTERESIS_MASK = 0xFFFF; // (16 bits)
572 static const uint32_t TRIG_DELAY_MASK = 0xFFFFFFFF; // (32 bits)
573 static const uint32_t WRITE_POINTER_MASK = 0x3FFF; // (14 bits)
574 static const uint32_t EQ_FILTER_AA = 0x3FFFF; // (18 bits)
575 static const uint32_t EQ_FILTER = 0x1FFFFFFF; // (25 bits)
576 static const uint32_t RST_WR_ST_MCH_MASK = 0x2; // (1st bit)
577 static const uint32_t TRIG_ST_MCH_MASK = 0x4; // (2nd bit)
578 static const uint32_t PRE_TRIGGER_COUNTER = 0xFFFFFFFF; // (32 bit)
579 static const uint32_t ARM_KEEP_MASK = 0x8; // (4 bit)
580 static const uint32_t FILL_STATE_MASK = 0x10; // (1 bit)
581 static const uint32_t TRIG_UNLOCK_MASK = 0x10; // (1 bit)
582
583 static const uint32_t AXI_ENABLE_MASK = 0x1; // (1 bit)
584 static const uint32_t AXI_CHA_FILL_STATE = 0x10; // (1 bit)
585 static const uint32_t AXI_CHB_FILL_STATE = 0x100000; // (1 bit)
586 static const uint32_t FULL_MASK = 0xFFFFFFFF; // (32 bit)
587
588 static const uint32_t DEBAUNCER_MASK = 0xFFFF; // (20 bit)
589
590 static const uint32_t CALIB_MASK = 0xFFFF; // (16 bit)
591
592 int osc_Init(int channels);
593 int osc_Release();
594
595 int osc_printRegset();
596
597 int osc_SetDecimation(rp_channel_t channel, uint32_t decimation);
598 int osc_GetDecimation(rp_channel_t channel, uint32_t* decimation);
599 int osc_SetAveraging(rp_channel_t channel, bool enable);
600 int osc_GetAveraging(rp_channel_t channel, bool* enable);
601 int osc_SetTriggerSource(rp_channel_t channel, uint32_t source);
602 int osc_GetTriggerSource(rp_channel_t channel, uint32_t* source);
603 int osc_SetSplitTriggerMode(bool enable);
604 int osc_GetSplitTriggerMode(bool* enable);
605 int osc_SetUnlockTrigger(rp_channel_t channel);
606 int osc_GetUnlockTrigger(rp_channel_t channel, bool* state);
607 int osc_WriteDataIntoMemory(rp_channel_t channel, bool enable);
608 int osc_ResetWriteStateMachine(rp_channel_t channel);
609 int osc_SetArmKeep(rp_channel_t channel, bool enable);
610 int osc_GetArmKeep(rp_channel_t channel, bool* state);
611 int osc_GetBufferFillState(rp_channel_t channel, bool* state);
612 int osc_GetTriggerState(rp_channel_t channel, bool* received);
613 int osc_GetPreTriggerCounter(rp_channel_t channel, uint32_t* value);

```

```

614 int osc_SetThresholdChA(uint32_t threshold);
615 int osc_GetThresholdChA(uint32_t* threshold);
616 int osc_SetThresholdChB(uint32_t threshold);
617 int osc_GetThresholdChB(uint32_t* threshold);
618 int osc_SetThresholdChC(uint32_t threshold);
619 int osc_GetThresholdChC(uint32_t* threshold);
620 int osc_SetThresholdChD(uint32_t threshold);
621 int osc_GetThresholdChD(uint32_t* threshold);
622 int osc_SetHysteresisChA(uint32_t hysteresis);
623 int osc_GetHysteresisChA(uint32_t* hysteresis);
624 int osc_SetHysteresisChB(uint32_t hysteresis);
625 int osc_GetHysteresisChB(uint32_t* hysteresis);
626 int osc_SetHysteresisChC(uint32_t hysteresis);
627 int osc_GetHysteresisChC(uint32_t* hysteresis);
628 int osc_SetHysteresisChD(uint32_t hysteresis);
629 int osc_GetHysteresisChD(uint32_t* hysteresis);
630 int osc_SetTriggerDelay(rp_channel_t channel, uint32_t decimated_data_num);
631 int osc_GetTriggerDelay(rp_channel_t channel, uint32_t* decimated_data_num);
632 int osc_GetWritePointer(rp_channel_t channel, uint32_t* pos);
633 int osc_GetWritePointerAtTrig(rp_channel_t channel, uint32_t* pos);
634 int osc_SetEqFilterBypass(rp_channel_t channel, bool enable);
635 int osc_GetEqFilterBypass(rp_channel_t channel, bool* enable);
636 int osc_SetEqFiltersChA(uint32_t coef_aa, uint32_t coef_bb, uint32_t coef_kk, uint32_t coef_pp);
637 int osc_GetEqFiltersChA(uint32_t* coef_aa, uint32_t* coef_bb, uint32_t* coef_kk, uint32_t* coef_pp);
638 int osc_SetEqFiltersChB(uint32_t coef_aa, uint32_t coef_bb, uint32_t coef_kk, uint32_t coef_pp);
639 int osc_GetEqFiltersChB(uint32_t* coef_aa, uint32_t* coef_bb, uint32_t* coef_kk, uint32_t* coef_pp);
640 int osc_SetEqFiltersChC(uint32_t coef_aa, uint32_t coef_bb, uint32_t coef_kk, uint32_t coef_pp);
641 int osc_GetEqFiltersChC(uint32_t* coef_aa, uint32_t* coef_bb, uint32_t* coef_kk, uint32_t* coef_pp);
642 int osc_SetEqFiltersChD(uint32_t coef_aa, uint32_t coef_bb, uint32_t coef_kk, uint32_t coef_pp);
643 int osc_GetEqFiltersChD(uint32_t* coef_aa, uint32_t* coef_bb, uint32_t* coef_kk, uint32_t* coef_pp);
644
645 int osc_SetCalibOffsetInFPGA(rp_channel_t channel, uint8_t bits, int32_t offset);
646 int osc_SetCalibGainInFPGA(rp_channel_t channel, double gain);
647 int osc_GetCalibOffsetInFPGA(rp_channel_t channel, uint8_t bits, int32_t* offset);
648 int osc_GetCalibGainInFPGA(rp_channel_t channel, double* gain);
649
650 int osc_SetExtTriggerDebouncer(uint32_t value);
651 int osc_GetExtTriggerDebouncer(uint32_t* value);
652
653 const volatile uint32_t* osc_GetDataBufferChA();
654 const volatile uint32_t* osc_GetDataBufferChB();
655 const volatile uint32_t* osc_GetDataBufferChC();
656 const volatile uint32_t* osc_GetDataBufferChD();
657
658 int osc_axi_EnableChA(bool enable);
659 int osc_axi_EnableChB(bool enable);
660 int osc_axi_EnableChC(bool enable);
661 int osc_axi_EnableChD(bool enable);
662

```

Protocolo - Header

Recordemos :

Contiene todo el contenido obligatorio de control sobre el resto de información para decodificar

```
struct __attribute__((__packed__)) header_t {  
    uint16_t version;      // Versión de protocolo.  
    api_id_t api_id;       // Mayor + minor.  
    uint32_t payload_size; // Bytes de texto que siguen al body.  
    uint8_t type;          // flag (REQUEST / RESPONSE / EVENT).  
};
```

```
enum flag : uint8_t {  
    REQUEST = 0,  
    RESPONSE,  
    EVENT, // Mensajes sin respuesta esperada (ej. CTRL_DEVICE_GONE).  
};
```

Características del protocolo:

- Tenemos un protocolo Binario sobre TCP.
- Cada paquete representa la llamada de una función de la API o un evento (Ej: Desconexión)
- API es un identificador de la función a llamar (tanto el cliente de hw y admin debe tener el mismo listado de funciones). Para esto tenemos el valor de versión.

API ID

```
116 #define API_VERSION 0x0001
117
118 struct __attribute__((__packed__)) api_id_t {
119     uint16_t family;           // api_family (Mayor)
120     uint16_t function_id;     // Minor (control_fn / system_fn / ...)
121 };
---
```

La identificación de las
✦ funciones tienen dos
números:

- Familia de funciones
- Número dentro de la familia

Funciona como un
major/minor

```
enum api_family : uint16_t {
    API_CONTROL      = 0, // mensajes del propio protocolo
    API_SYSTEM       = 1, // rp_Init, rp_Release, rp_GetVersion
    API_ACQUISITION  = 2, // rp_Acq* (osciloscopio)
    API_GENERATION    = 3, // rp_Gen*
    API_DIGITAL_IO    = 4, // rp_Dpin*
    API_ANALOG_IO     = 5, // rp_Apin*
    API_HW_PROFILE    = 6, // hp_cmnn_Print: dump del profile del board
};
```

Mayors

```
// Minor para los mensajes de control.
enum control_fn : uint16_t {
    CTRL_HELLO = 0, // Hardware/Admin -> Control: "soy <mac>"; resp: accept
    CTRL_LIST_DEVICES, // Admin -> Control; resp: lista de dispositivos
    CTRL_DEVICE_GONE, // Control -> Admin (EVENT): se desconectó un device
    CTRL_BYE, // Hardware/Admin -> Control: me voy
    STD_OUTDEVICE, // Hardware -> Control -> Admin: stdout del programa
    NUM_CONTROL_CMD,
};
```

Ej Minor: API_CONTROL ()

Body

Esto refiere a todos el contenido **condicional**, o sea que depende del contenido del header.

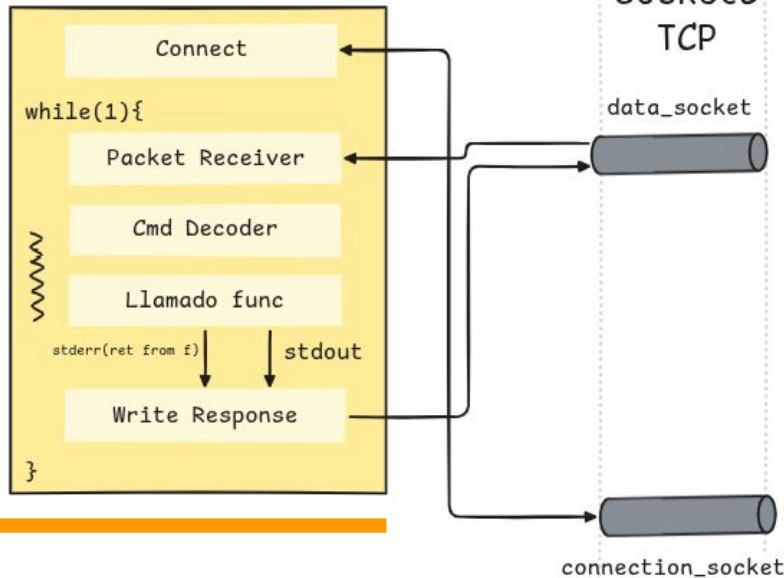
En este caso el tipo de paquete determina el tipo de body a utilizar.

- Body del request:
Argumento de la función
- Body del Response:
retval (condición de error)
stdout del programa original: Posiblemente extenso
- Body del Event:
Raw dependiente del event

```
struct __attribute__((__packed__)) body_t {  
    device_mac destination_device; // A qué device va dirigido (0 = control).  
    uint32_t request_id;          // Lo asigna el Admin/Control para correlar.  
    uint32_t origin_admin;        // admin_id que originó la llamada.  
    int32_t retval;                // Valor de retorno (en RESPONSE).  
    union {  
        int32_t i1;  
        double d1;  
        struct __attribute__((__packed__)) {  
            int32_t a;  
            int32_t b;  
        } ii;  
        uint8_t raw[16];  
    } params;  
};
```

Cliente de Hardware

Hardware



1. En cada HW vamos a tener un proceso que va a conectarse al controlador a una dirección fija y conocida (connection_socket).
2. En esta conexión se crea otro ✨ sockets de transferencia de datos.
3. Tenemos un while que espera por comandos, ejecuta, captura la salida del programa y la pasa como un paquete tipo RESPONSE.

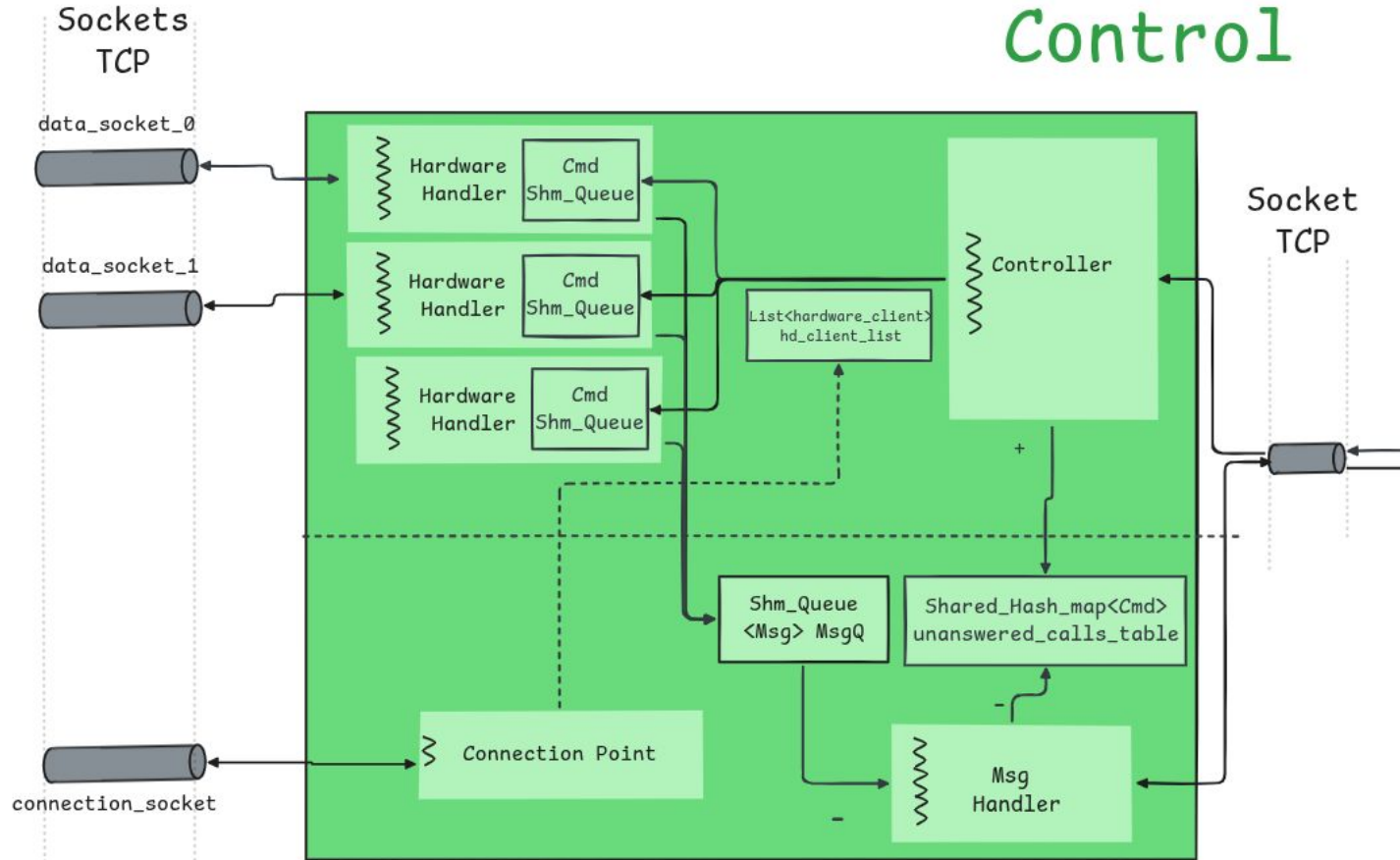
Llamada a API

Mega-Case sobre API-ID
con dos niveles

Usamos un *union params*
para hacer un “cast” sobre
los valores binarios del
paquete.

```
static int call_rp_function(const cmd_t& cmd, std::string* out_text) {  
    switch (cmd.header.api_id.family) {  
        case API_SYSTEM:  
            switch (cmd.header.api_id.function_id) {  
                case SYS_INIT:  
                    // return rp_Init();  
                    printf("rp_Init() llamado");  
                    return 0;  
                case SYS_RELEASE:  
                    // return rp_Release();  
                    printf("rp_Release() llamado");  
                    return 0;  
                case SYS_RESET:  
                    // return rp_Reset();  
                    printf("rp_Reset() llamado");  
                    return 0;  
                case SYS_GET_VERSION:  
                    // *out_text = rp_GDDetVersion();  
                    *out_text = "rp-stub-1.0.0";  
                    return 0;  
            }  
            break;  
        case API_ACQUISITION: {  
            // Lazy init: una sola vez por proceso. cmn_Map del shim es  
            // idempotente, así el state del osc_reg sobrevive entre set/get.  
            static bool osc_initd = false;  
            if (!osc_initd) { osc_Init(1); osc_initd = true; }  
  
            const auto ch_a = (rp_channel_t)cmd.body.params.ii.a;  
            const auto ch_i = (rp_channel_t)cmd.body.params.i1;  
            uint32_t v32 = 0;  
            bool      vb = false;  
        }  
    }  
}
```

Control



Threads de Control

Connection Point

`accept() de hw_clients`

`Create hardware_client`

`Add to hd_client_list`

`Create Data Socket
and Cmd_queue`

`Span Hardware handler`

Msg Handler

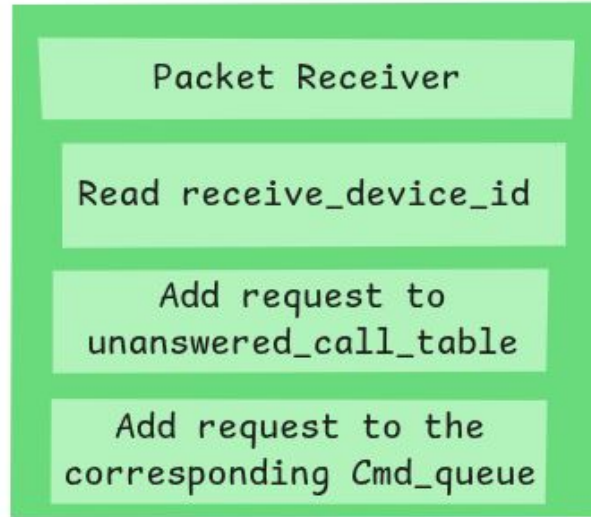
`Take from Msg_queue`

`Fetch request src from
unanswered_call_table`

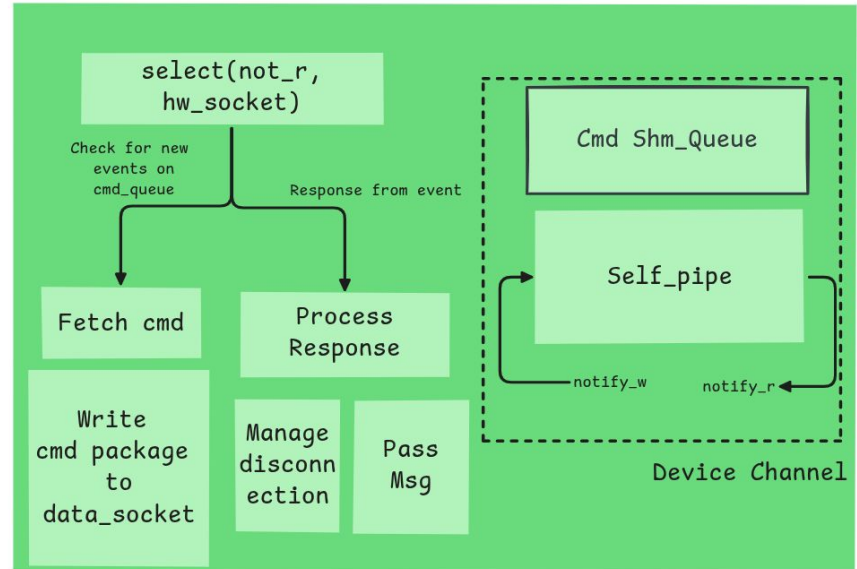
`Push response to Admin`

Resto de Subprocesos.

Controller



Hardware Handler

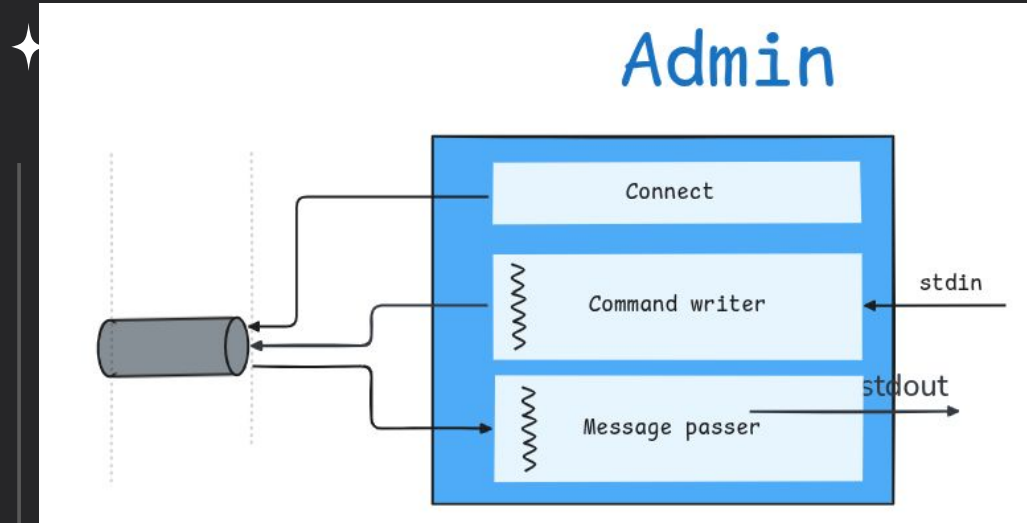


Admin:

Este es el elemento más sencillo.

Hace dos cosas

1. Primer conecta con Control por una dirección fija y conocida
2. Spanwea dos hilos independientes (hace detach)



Command Writer:

1. lee comandos línea por línea (de un archivo o de stdin)
2. Parsea las líneas en busca y crea cmd
3. Manda al Control

Message Parser:

Hilo que lee el socket de llegada e Imprime a consola.

Proceso de Compilación

El proceso de compilación es más complejo que un simple `g++ <archivos>`. Tengo que linkear sobre funciones internas de la API que dependen de drivers específicos del OS del a Pitaya

```
int osc_Init(int channels) {  
    ECHECK(cmn_Map(OSC_BASE_SIZE, OSC_BASE_ADDR, (void*)&osc_reg))  
    osc_cha = (uint32_t*)((char*)osc_reg + OSC_CHA_OFFSET);  
    osc_chb = (uint32_t*)((char*)osc_reg + OSC_CHB_OFFSET);  
  
    if (channels == 4) {  
        size_t base_addr = OSC_BASE_ADDR_4CH;  
        ECHECK(cmn_Map(OSC_BASE_SIZE, base_addr, (void*)&osc_reg_4ch))  
        osc_chc = (uint32_t*)((char*)osc_reg_4ch + OSC_CHA_OFFSET);  
        osc_chd = (uint32_t*)((char*)osc_reg_4ch + OSC_CHB_OFFSET);  
    }  
  
    return RP_OK;  
}
```

Necesita



```
int cmn_Init() {  
    if (fd == -1) {  
        if ((fd = open("/dev/uio/api", O_RDWR | O_SYNC)) == -1) {  
            return RP_EOMD;  
        }  
    }  
  
    return RP_OK;  
}
```

Para acceder a la API desde mi computadora tengo que compilar objetos

Solución: Precompilar Archivos Objetos

Compilo parte de la API (elementos básicos)
para mi computadora

- Ver Makefile
- Mostrar que el linkeo con `nm` y `nm -C`.

```
RP_INC      = -IRedPitaya/rp-api/api/include \  
            -IRedPitaya/rp-api/api/src \  
            -IRedPitaya/rp-api/api/include/common \  
            -IRedPitaya/rp-api/api-hw-profiles/include \  
            -IRedPitaya/rp-api/api-hw-profiles/src \  
            -IRedPitaya/rp-api/api-hw-calib/include  
RP_CXXFLAGS = -std=c++20 -O2 -w -pthread -ffunction-sections -fdata-sections  
RP_CFLAGS   = -std=c11 -O2 -w -pthread -ffunction-sections -fdata-sections  
RP_LDFLAGS   = -Wl,--gc-sections
```

```
(base) lorenzo@LoloVictus:~/Desktop/IB/Sexto_Semestre/Lab_Redex/TP_Final/build$ nm -C -a hardware_client | grep cmn  
000000000000bb80 T hp_cm_n_GetADCBaseRateFromConfig  
000000000000367e t hp_cm_n_GetADCBaseRateFromConfig.cold  
000000000000c5d0 T hp_cm_n_GetDACBaseRateFromConfig  
000000000000372d t hp_cm_n_GetDACBaseRateFromConfig.cold  
000000000000aef0 T hp_cm_n_Print  
0000000000000000 a mock_cm_n.cpp  
00000000000052d0 T cm_n_InitMap(unsigned long, unsigned long, void**, int*)  
0000000000004e70 T cm_n_GetValue(unsigned int volatile*, unsigned int*, unsigned int)  
0000000000004e90 T cm_n_SetValue(unsigned int volatile*, unsigned int, unsigned int, unsigned int*)  
0000000000004e50 T cm_n_ReleaseClose(int, unsigned long, void**)  
0000000000004e60 T cm_n_isEnableDebugReg()  
000000000000ae30 T hp_cm_n_GetHomeDirectory[abi:cxx11]()  
00000000000052b0 T cm_n_Map(unsigned long, unsigned long, void**)
```


Demostración

Hice varios archivos de **test end-to-end**. Crean

- Control:
 - Asigna puertos
- Admin:
 - Indica puertos
- ✦ - Hardware client: Puertos 9101-
 - Da valores de MAC (ej: AABBCCDDEEFF)
 - Indica puertos

En los siguientes

- e2e.sh: 1 HW.
 - E2e_multi.sh: 3 HW
 - e2e-interactive.sh: 3 HW con intérprete de comandos.
- ✦

Un testeo más integral tendría en cuenta test unitarios sobre cada uno de los bloques (hw_client, control y admin). O hasta un testeo unitario por bloques específicos, particularmente para el bloque de control.

Conclusiones

- El Control Server elimina la complejidad de administrar N conexiones simultáneas, reduciendo la interfaz del operador a un único punto de control para todo el cluster.
- Se comunicó de forma exitosa todos los componentes en la pruebas end-to-end.
- Se implementó un modelo híbrido request-response + eventos.

Github

Para los que quieran ver el proyecto se encuentra en público en :
https://github.com/LorenBataraza/RedPitaya_Control_Server

The screenshot shows the GitHub repository page for **RedPitaya_Control_Server** by user **LorenBataraza**. The repository is public and has 0 stars, 0 forks, and 0 watchers. It is currently on the **main** branch with 1 branch and 0 tags. The repository description is "No description, website, or topics provided." The file list includes:

File/Folder	Description	Time
RedPitaya @ 091fe57	Skeleton	3 weeks ago
include	Added API Mockup with Cmn object injection	4 days ago
tests	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
.gitignore	Test scripts (single + multi device), build/ output dir, Devic...	2 weeks ago
.gitmodules	Skeleton	3 weeks ago
Makefile	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
admin_cmd_reader.cpp	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
admin_cmd_reader.h	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
admin_main.cpp	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
api.cpp	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
control_server.cpp	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
hardware_client.cpp	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago
mock_cmn.cpp	Add: Hardware Profiles, Added Original API Symbols and ...	11 minutes ago

The right sidebar shows the **About** section with the repository description, **Releases** section with "No releases published" and a link to "Create a new release", **Packages** section with "No packages published" and a link to "Publish your first package", **Contributors** section with 1 contributor (LorenBataraza), and the **Languages** section.



Muchas gracias